

Code review — recapitulare-cpp

Runde de review pe proiectul tău, cea mai nouă sus.

Runda 1 — 2026-09-24 — sha f7a9943

Revizuit: `recapitulare/rezolvare-fisa-2.h` (exercițiile 1–19 din fișa 2), `recapitulare/lucrul-cifre.h`, `recapitulare/recapitulare.cpp`.

Poarta de compilare:  **trece.** `g++ -std=c++20 -Wall -Wextra` compilează fără erori, cu 2 warning-uri (ambele apar mai jos).

Verificarea s-a făcut prin rulare, nu prin citire. Am copiat cele două headere într-un proiect separat și am chemat fiecare funcție pe valorile din fișă plus pe cele patru din checklist-ul final (`7`, `0`, `4038`, întoarce sau afișează). 60 de apeluri: **48 corecte, 12 greșite.**

Corecte cap-coadă: exercițiile **2, 4, 6, 7, 8, 9, 13, 17**. Ex. 17 (*Fără cifrele pare*) e cel mai greu din Partea 3 și e scris corect, cu `p` care ține poziția ca să păstreze ordinea — exact ideea cerută. Ex. 13 respectă convenția „poziții numărate de la dreapta”, care era ușor de citit greșit. Iar `eliminareaCifreiDeRangk` din `lucrul-cifre.h` (lucrul de la clasă) trece toate cele 4 teste pe care i le-am dat, inclusiv cu `0` în mijlocul numărului.

Și un lucru care nu se vede în rezultate: deasupra lui `eliminareaCifreiDeRangk`, la `lucrul-cifre.h:97–103`, ți-ai scris tabelul de urmărire pas cu pas (`n`, `uc`, `nou`, `p`). Ți-a dat reflexul care repară singur jumătate din bug-urile de mai jos. Fă-l și la celelalte.

Critice

B1 — `apareCifra` răspunde la altă întrebare

`rezolvare-fisa-2.h:49`

CUM E ACUM: `apareCifra(4738, 7)` întoarce `false`. Fișa cere `true`. Funcția e adevărată doar când **toate** cifrele sunt `x`: `apareCifra(777, 7) → true`, dar `apareCifra(4738, 7) → false`.

DE CE: ai luat forma din `isCifreIdentice` și ai schimbat doar ce se compară. Dar forma aia — `return false` în buclă, `return true` după — e răspunsul la „toate?”. Bucla caută un contraexemplu și, dacă nu-l găsește până la capăt, declară adevărul.

Întrebarea „există?” are exact forma oglindită: bucla caută o **dovadă**, `return true` stă în buclă, iar `return false` stă **după** ea, pentru că abia atunci știi că n-ai găsit nimic. Cele două tipare nu se pot amesteca: cu `return` pe prima cifră care nu e `x`, funcția ta se oprește la `8` din `4738` și nu mai apucă niciodată să vadă `7` -ul.

Regula de verificat singură: după ce scrii bucla, întreabă-te ce *dovedește prima ieșire din ea*. La „toate”, prima ieșire e o infirmare. La „există”, prima ieșire e o confirmare.

FIX: `if (cif == x) { return true; }` în buclă, `return false;` după ea.

B2 — `while (n > 10)` se oprește cu o cifră prea devreme

`rezolvare-fisa-2.h:7` (ex. 1) · același bug la `:179` (ex. 14)

CUM E ACUM:

apel	rezultat	corect
<code>primaCifra(105)</code>	<code>10</code>	<code>1</code>
<code>primaCifra(100)</code>	<code>10</code>	<code>1</code>
<code>primaCifra(10)</code>	<code>10</code>	<code>1</code>
<code>alipirePrimaUltima(105)</code>	<code>105</code>	<code>15</code>
<code>alipirePrimaUltima(10)</code>	<code>100</code>	<code>10</code>

DE CE: bucla trebuie să taie cifre **cât timp mai e mai mult de una**, adică atâta timp cât `n` are două sau mai multe cifre. „Are cel puțin două cifre” se scrie `n >= 10`, nu `n > 10` — pentru că `10` este primul număr de două cifre, nu ultimul de o cifră.

Cu `n > 10`, când `n` ajunge fix la `10` condiția devine falsă și bucla se oprește cu `n = 10` în mână. Funcția întoarce `10` ca și cum ar fi o cifră. De-aia `4738` iese corect (coborârea trece prin `473`, `47`, `4` și nu atinge niciodată `10`), iar `105` iese greșit (`105 → 10`, stop).

La ex. 14 e aceeași linie copiată, deci același bug: `alipirePrimaUltima(10)` calculează $10 * 10 + 0 = 100$.

FIX: `while (n >= 10)` în ambele locuri.

B3 — `radacinaDigitala` nu reinițializează `s`, iar numărul crește până depășește `int`

rezolvare-fisa-2.h:207

CUM E ACUM: `radacinaDigitala(9875)` întoarce `-2147483615`. Fișa cere `2`.

`radacinaDigitala(10)` întoarce `0`, corect ar fi `1`.

DE CE: `s` e declarat la `:208`, în afara buclei exterioare. Buclea interioară adună cifrele **peste** suma rămasă din pasul anterior, în loc să pornească de la zero. La fiecare tur suma e tot mai mare, deci `n = s` de la `:215` nu scade, ci crește:

`9875 → s=29` · apoi `29 → s=29+9+2=40` · apoi `40 → s=40+0+4=44` · apoi `44 → 52 → 59 → 73 → ...`

Rădăcina digitală funcționează **numai** pentru că suma cifrelor e mereu mai mică decât numărul. Ai rupt fix garanția asta, așa că procesul nu mai are unde să se oprească.

Și atunci de ce programul nu îngheață, ci îți dă un număr? Pentru că `s` crește până trece de `2147483647`, maximul lui `int`. La depășire valoarea se rostogolește în negativ, condiția `n > 10` devine falsă din alt motiv decât cel pe care-l voiai, și funcția întoarce gunoiul. Asta e cel mai periculos fel de bug: **nu crapă, îți răspunde.**

FIX: mută `int s = 0;` **înăuntrul** buclei exterioare, ca să repornească de la zero la fiecare tur. Sau, mai scurt: cheamă `sumaCifrelor(n)` (B10).

B4 — `while (n > 0)` nu rulează niciodată pentru `n = 0`

rezolvare-fisa-2.h:27 (ex. 3) · **același bug la** `lucrul-cifre.h:24`, `:35`, `:78`

Restanță: constatarea asta era deschisă și la analiza din 17 septembrie.

CUM E ACUM:

apel	rezultat	corect
<code>produsulCifrelor(0)</code>	1	0
<code>contorCifre(0)</code>	0	1
<code>contorCifrePare(0)</code>	0	1
<code>ciframinima(0)</code>	10000	0

DE CE: `while` își testează condiția **înainte** de prima trecere. Pentru `n = 0`, `0 > 0` e fals din start, deci corpul buclei nu rulează niciun tur. Funcția întoarce valoarea cu care ai inițializat variabila — 1 la produs, 0 la contoare, 10000 la minim — și niciuna dintre ele nu e un răspuns, sunt puncte de plecare rămase neatinse.

Problema de fond: 0 **are** o cifră. Orice buclă construită pe „cât timp mai am din ce tăia” tratează 0 ca pe un număr fără cifre. De-aia checklist-ul din fișă îți cerea explicit să treci fiecare funcție prin 0 — cârligul era acolo.

`do...while` ar rezolva-o dintr-o mișcare, dar e pe lista interzisă deocamdată. Cu ce știi acum, fixul e o linie de gardă înainte de buclă.

FIX: `if (n == 0) { return 0; }` la produs, `return 1;` la contoare, `return 0;` la `ciframinima`.

B5 — `cifreComune` citește în afara vectorului

`rezolvare-fisa-2.h:256`

CUM E ACUM: `int ctn[10]` are indici valizi de la 0 la 9. Bucla merge `while (i <= 10)`, deci la ultimul tur citește `ctn[10]` și `ctm[10]` — în afara vectorilor. Rulat cu AddressSanitizer:

```
ERROR: AddressSanitizer: stack-buffer-overflow
READ of size 4 ... in cifreComune(int, int) rezolvare-fisa-2.h:257
'ctn' (line 241) <== Memory access at offset 72 overflows this variable
```

DE CE: un vector de 10 elemente are ultimul element pe poziția 9, nu 10 — numărătoarea pornește de la 0. C++ **nu verifică** dacă indicele încape; pur și simplu citește cei 4 octeți de după sfârșitul vectorului. Acolo nu e nimic al tău, e ce s-a întâmplat să stea alături în memorie.

Partea care trebuie să te sperie: `cifreComune(4038, 2065)` îmi întoarce `1`, care **este** răspunsul corect. Nu pentru că funcția e bună, ci pentru că `ctn[10]` nimereste exact peste `ctm[0]`, vecinul din memorie, și acolo se întâmplă să fie un `1`. Un test care trece nu dovedește nimic dacă programul citește memorie care nu-i aparține — mâine, cu alt compilator sau cu altă ordine a variabilelor, răspunsul se schimbă singur.

FIX: `while (i <= 9)`.

B6 — numărătoarea cifrelor pornește de la `i = 1`, deci cifra `0` nu există niciodată

`rezolvare-fisa-2.h:195` (ex. 15) · același bug la `:254` (ex. 18) · și, în oglindă, la `:275` (ex. 19)

CUM E ACUM:

apel	rezultat	corect
<code>ctCifDif(4038)</code>	<code>3</code>	<code>4</code>
<code>nrMax(1203)</code>	<code>321</code>	<code>3210</code>
<code>nrMax(1000)</code>	<code>1</code>	<code>1000</code>

DE CE: în `ct[10]`, poziția `ct[0]` ține **de câte ori apare cifra 0**. E o casetă la fel de validă ca `ct[7]`. Când pornești baleierea de la `i = 1`, sari peste ea — nu pentru că `0` n-ar fi cifră, ci pentru că ai confundat „indicele de start al unui vector” cu „prima cifră care mă interesează”.

Se vede cel mai clar la `4038`: îl numeri o dată pe `4`, o dată pe `3`, o dată pe `8`, iar pe `0` îl scrii în `ct[0]` — și apoi nu te mai uiți acolo. `4038` era fix cazul din checklist-ul fișei („un `0` pe la mijloc”).

FIX: `int i = 0;` la ex. 15 și ex. 18. La ex. 19 baleierea merge invers, de la `9` în jos, deci acolo capătul lipsă e celălalt: `while (i >= 0)`. Și verifică B7 după — în `nrMax` sunt două bug-uri diferite, iar `nrMax(1000)` nu se repară decât cu amândouă.

B7 — `nrMax` folosește fiecare cifră o singură dată, oricâte are numărul

`rezolvare-fisa-2.h:276`

CUM E ACUM: `nrMax(4438)` întoarce `843`. Corect: `8443`. Un `4` se pierde pe drum. (`nrMax(4738)` iese corect fiindcă acolo n-ai cifre repetate — de-aia bug-ul nu s-a văzut pe exemplul din fișă.)

DE CE: în `ct[10]` ai pus un **contor**, dar la `:276` îl citești ca pe un **bifat / nebifat**: `if (ct[i] != 0)` întreabă doar „a apărut măcar o dată?” și, dacă da, lipește cifra **o singură dată**. Informația „de câte ori” e acolo, în vector — doar că n-o folosești.

Ca să construiești cel mai mare număr, fiecare cifră trebuie lipită de atâtea ori câte apariții are: un `for` interior care se învâрте `ct[i]` ori.

FIX: în locul lui `if`, o buclă interioară care repetă `x = i + x * 10;` de `ct[i]` ori.

B8 — `cifCrescatoare` acceptă cifre egale, deși cerința zice „strict”

`rezolvare-fisa-2.h:141`

CUM E ACUM: `cifCrescatoare(1155)` întoarce `true`. Fișa are exemplul scris explicit: `1155 -> false`.

DE CE: compari cu `if (cif < n % 10)`, care declară „nu e crescător” **numai** când pasul coboară strict. Când cele două cifre sunt egale, condiția e falsă, bucla merge mai departe și perechea `5 5` trece nesancționată.

„Strict crescătoare” înseamnă că fiecare pas trebuie să **urce**, deci orice pas care nu urcă — și coborârea, și egalitatea — trebuie respins. Diferența dintre `<` și `<=` e exact diferența dintre „crescătoare” și „strict crescătoare”; aici cerința o cere pe a doua.

FIX: `if (cif <= n % 10)`.

B9 — `pozitieCifra` are o ieșire fără `return`

`rezolvare-fisa-2.h:85-89`

CUM E ACUM: compilatorul îți spune direct:

```
rezolvare-fisa-2.h:89: warning: non-void function does not return
a value in all control paths
```

Reprodus cu `pozitieCifra(4738, 0)` : compilat cu `-O0` întoarce `1` , compilat cu `-O1` întoarce `68981787` . Aceeași funcție, aceeași intrare, alt răspuns. (Cu pozițiile din fișă — `x` de la `1` în sus — *n-am reușit să-l scot; apare doar pentru `x <= 0` .)*

DE CE: când o funcție `int` se termină fără `return` , nu primești zero și nu primești eroare — apelantul citește pur și simplu ce s-a nimerit în locul de unde se ia valoarea întoarsă. De-aia rezultatul se schimbă cu nivelul de optimizare: nu e o valoare, e o rămășiță.

Ultimul `if (ct < x)` de la `:85` e de fapt inutil. Dacă bucla s-a terminat fără să întoarcă nimic, înseamnă că n-a existat poziția cerută — nu mai e nimic de testat, răspunsul e `-1` necondiționat. `if` -ul acela nu adaugă o verificare, doar lasă o ieșire descoperită.

FIX: șterge `if` -ul și lasă `return -1;` simplu.

B10 — corpurile funcțiilor stau în `.h`

`rezolvare-fisa-2.h` (tot fișierul) · `lucrul-cifre.h` (tot fișierul)

Restanță: deschisă și la analiza din 17 septembrie; acum e în două fișiere.

CUM E ACUM: merge, pentru că proiectul are un singur `.cpp` . Am adăugat un al doilea `.cpp` care include același header și link-editarea cade:

```
duplicate symbol 'cifCrescatoare(int)' in: al2lea.o, mainlink.o
duplicate symbol 'sumaPozImpr(int)' in: al2lea.o, mainlink.o
duplicate symbol 'sumaCifrelorPare(int)' ...
```

DE CE: `#pragma once` te apără de includerea aceluiași header de două ori **în aceeași unitate de compilare**. Dar fiecare `.cpp` se compilează separat, iar `#include` înseamnă „copiază tot textul aici”. Două `.cpp` care includ headerul primesc fiecare **câte o copie completă** a celor 19 funcții, și fiecare obiect rezultat conține definițiile. Linkerul, care le pune apoi cap la cap, găsește două definiții pentru același nume și refuză.

Împărțeala corectă: în `.h` doar **promisiunea** `( int sumaCifrelor(int n);` — antetul și punct-virgulă), în `.cpp` **corpul**. Promisiunea se poate copia de câte ori vrei; corpul trebuie să existe o singură dată.

FIX: `rezolvare-fisa-2.cpp` cu corpurile, `rezolvare-fisa-2.h` doar cu cele 19 antete. Nu e urgent cât timp ai un `.cpp` , dar e prima chestie care sparge proiectul când îl faci mai mare.

🟡 Importante

M1 — 19 funcții noi, zero apeluri către funcțiile tale. Am căutat în `rezolvare-fisa-2.h` : nicio funcție nu cheamă altă funcție. Fișa era ordonată special ca să nu fie nevoie să rescrii bucla:

exercițiu	ce ai scris	ce aveai deja
9 palindrom :107	bucla de răsturnare, din nou	<code>rasturnat(n)</code> de la ex. 8
16 rădăcina digitală :210	bucla de sumă, din nou	<code>sumaCifrelor(n)</code> de la ex. 2
14 prima și ultima :179	bucla de tăiere, din nou	<code>primaCifra(n)</code> de la ex. 1
15, 18, 19	vector de contori	<code>contorAparitie(n, x)</code> de la ex. 6

Și costul nu e doar de scris în plus: bug-ul B2 există în **două** locuri tocmai pentru că ai copiat bucla din ex. 1 în ex. 14. Dacă ex. 14 chema `primaCifra`, reparai o linie și se reparau amândouă.

În septembrie ai scris `contorCifreImpare` ca `contorCifre(n) - contorCifrePare(n)` — exact reflexul care trebuie. Pe fișa 2 a dispărut. Țsta e lucrul de recuperat, nu bug-urile.

M2 — vectorii sunt pe lista interzisă a fișei, i-ai folosit în 4 exerciții. `int ct[10]` la :123 , :188 , :241 , :242 , :267 (ex. 10, 15, 18, 19). Nu e o chichiță de regulament: vectorul a fost exact scurtătura care te-a ferit de reutilizare (M1) și care a adus B5 și B6. Cele patru se pot face cu `contorAparitie` și două bucle.

M3 — ex. 20 nu e făcut. Commit-ul zice „ex 1-19”, deci știi. E singurul din fișă care cere cod în `main()` și singurul care te pune să chemi `isCifreIdentice` -ul tău vechi.

M4 — `main()` = 90 de linii de teste comentate, `recapitulare.cpp:6-96`. Testezi câte o funcție, apoi comentezi apelul și treci la următoarea. După ce repari B2, n-ai cum să verifici dintr-o rulare că celelalte 18 au rămas întregi — ar trebui să decommentezi 19 blocuri pe rând. Scrie `main()` cu toate apelurile active și cu `cout` care spune ce compară: `cout << "ex1 " << primaCifra(105) << "(astept 1)" << endl;`

M5 — `repetareCifra` (:121) se cheamă invers față de ce face. Întoarce `true` când cifrele **nu** se repetă. Cine citește `if (repetareCifra(n))` va înțelege „are cifre repetate” și va scrie codul pe dos. Numele unei funcții `bool` trebuie să fie afirmația care e adevărată când întoarce `true` : `areCifreDistincte` .

M6 — `alipire(12, 0)` întoarce `12`, nu `120`. `rezolvare-fisa-2.h:153` — aceeași cauză ca B4: pentru `m = 0` bucla nu rulează, `p` rămâne `1`. Fișa spune că numerele sunt pozitive **sau** `0`, deci cazul intră în cerință.

Cleanups

- **C1** — `using namespace std;` în header (`lucrul-cifre.h:3` , `rezolvare-fisa-2.h:3`). Într-un `.h` îl impui oricui îl include.
 - **C2** — `lucrul-cifre.h:25` : `int cif = n % 10;` nefolosit în `contorCifre` . E singurul warning pe care `-Wall` îl dă în fișierul vechi.
 - **C3** — `recapitulare/.gitignore.txt` — fișier gol, comis din greșeală.
 - **C4** — numele funcțiilor merg pe trei stiluri: `ciframaxima` , `contorCifrePare` , `ctCifDif` . Alege unul (`camelCase`) și scrie cuvinte întregi: `contorCifreDiferite` , nu `ctCifDif` .
 - **C5** — `lucrul-cifre.h:53` : `cout << "n=" << endl;` — `endl` -ul mută cursorul pe rândul următor, deci scrii `n=` și tastezi dedesubt. La `ex2()` ai scris-o corect, fără `endl` .
 - **C6** — ambele headere se termină fără linie nouă la final.
-

Before / After

#	Acum	Corect
B1	<code>if (cif != x) return false; ... return true;</code>	<code>if (cif == x) return true; ... return false;</code>
B2	<code>while (n > 10)</code>	<code>while (n >= 10)</code>
B3	<code>int s = 0;</code> înainte de <code>while (n > 10)</code>	<code>int s = 0;</code> ca primă linie în <code>while (n > 10)</code>
B4	<code>int P = 1;</code> apoi <code>while (n > 0)</code>	<code>if (n == 0) return 0;</code> înainte de buclă
B5	<code>while (i <= 10)</code>	<code>while (i <= 9)</code>
B6	ex. 15/18: <code>int i = 1;</code> · ex. 19: <code>while (i > 0)</code>	ex. 15/18: <code>int i = 0;</code> · ex. 19: <code>while (i >= 0)</code>
B7	<code>if (ct[i] != 0) { x = i + x * 10; }</code>	<code>int j = 0; while (j < ct[i]) { x = i + x * 10; j++; }</code>
B8	<code>if (cif < n % 10)</code>	<code>if (cif <= n % 10)</code>
B9	<code>if (ct < x) { int a = -1; return a; }</code>	<code>return -1;</code>
M1	bucla de răsturnare rescrisă în <code>palindrom</code>	<code>return rasturnat(n) == n;</code>
M6	<code>int p = 1;</code> apoi <code>while (m > 0)</code>	<code>if (m == 0) { return n * 10; }</code> înainte de buclă

Întrebări

1. La B1, `apareCifra(777, 7)` întoarce `true` și `apareCifra(4738, 5)` întoarce `false` — ambele răspunsuri corecte, cu o funcție greșită. De ce reușesc fix astea două, și ce au în comun?
2. La B5 ți-am arătat că `cifreComune(4038, 2065)` dă `1`, adică răspunsul bun, deși citește în afara vectorului. Dacă un test trece, dar programul citește memorie care nu-i aparține — ce anume ai dovedit cu testul ăla?
3. La B3, dacă `s` s-ar reseta corect, de ce e sigur că bucla exterioară se oprește mereu, oricât de mare ar fi numărul de la care pleci? Ce proprietate a sumei cifrelor garantează asta?



MyCodeSchool

Material didactic MyCodeSchool

mycodeschool.ro